

Lab5 Report

Name: 박수현

Student ID: 2022063672

수업 번호: 12916

1. Overall Structure

HW4와 비교하여 HW5에서 변경된 점은 다음과 같다.

단계 사이에 파이프라인 레지스터 추가

CTRL.v에서 FSM 제거 - 명령어 디코더로 변경

HAZARD.v 신규 추가 - 데이터 해저드 검출, stall, flush 처리

branch not taken 예측 사용

branch: MEM stage에서 taken 판정 - Taken일 경우 IF/ID, ID/EX, EX/MEM 3개 flush

jump는 ID 단계에서 조기 결정 - Taken일 경우 IF/ID 1개 flush

2. Pipeline Stages

파이프라인 CPU는 하나의 명령어를 5단계로 나누어 수행하며, 각 단계는 매 사이클마다 서로 다른 명령어를 동시에 처리한다. branch not taken 예측을 사용하므로, 분기가 taken일 때만 잘못 fetch된 명령어를 flush한다. 각 단계별 수행 내용은 다음과 같다.

IF

- PC가 가리키는 주소에서 명령어를 fetch하여 IF/ID 파이프라인 레지스터에 저장한다.

- $if_pc_plus4 = PC + 4$ 를 계산하여 함께 저장한다.

- PC_next는 branch_taken과 jump_taken 의 값에 따라 결정된다. Branch 명령어가 더 높은 명령어이므로 branch 판정 이후 jump를 판정한다.

ID

- IF/ID 레지스터에 저장된 명령어를 디코딩하고 RF에서 rs, rt 값을 읽는다.
- CTRL 모듈이 opcode/funct를 디코딩하여 전 단계 제어신호를 한 번에 생성한다.
- J/JAL/JR의 경우 이 단계에서 jump 주소를 결정하고 id_jump_taken을 assert한다. JAL 명령어의 경우 WB 스테이지에서 ra 레지스터 쓰기를 진행한다.
- HAZARD 모듈이 데이터 해저드를 검출하면 파이프라인을 stall한다.

EX

- ALU 연산을 수행한다. operand1은 항상 idex_rd_data1, operand2 = ALUSrc ? idex_ext_imm : idex_rd_data2.
- branch 타겟 주소 계산
- WB에서 쓸 레지스터 주소 결정.

MEM

- LW의 경우 exmem_alu_result 주소에서 데이터를 읽는다.
- SW의 경우 exmem_rd_data2를 해당 주소에 저장한다.
- 분기 결정: mem_branch_taken = exmem_Branch && exmem_alu_result.
- branch taken 시 IF/ID, ID/EX, EX/MEM을 모두 flush한다.

WB

- RF에 결과를 기록한다. SavePC ? pc_plus4 : (MemtoReg ? mem_data : alu_result).
- RF 내부 포워딩으로, 같은 사이클에 WB 쓰기와 ID 읽기가 겹치면 새 값이 즉시 반환된다.

3. Implementation

- CTRL.v

멀티사이클의 FSM을 제거하고, HW3 스타일의 명령어 디코더로 대체하였다. always 블록 앞에서 모든 제어신호를 0으로 초기화한 뒤, 각 opcode별로 필요한 신호만 설정한다. 제어신호는 한 사이클에 해당 명령어의 전 단계(EX/MEM/WB) 제어신호를 동시에 생성하며, 중간 단계 레지스터가 이를 단계별로 전달한다.

멀티사이클 전용 신호를 삭제하고, 파이프라인용 신호를 새로 추가하였다. Branch는 BEQ/BNE일 때 1로 설정되며, MEM 단계에서 PCSrc = Branch & ALU_zero로 분기 여부를 결정한다.

- CPU.v

멀티사이클의 state 레지스터 및 중간 latch 를 제거하고, 파이프라인 레지스터 4개(IF/ID, ID/EX, EX/MEM, MEM/WB)를 추가하였다. 각 레지스터는 clock의 positive edge에서 이전 단계의 결과를 저장한다.

PC 갱신 및 플러시 로직은 우선순위에 따라 동작한다. branch_taken이 최우선이고, 다음으로 jump_taken, 마지막으로 PC+4를 사용한다. branch taken 시에는 IF/ID, ID/EX, EX/MEM을 flush하고, jump taken 시에는 IF/ID만 flush한다. 단, PCWrite=0인 stall 중에는 flush가 일어나지 않도록 id_flush_if = id_jump_taken & PCWrite 조건을 적용한다. branch taken은 stall을 무시하고 PC를 갱신해야 하므로 PC 갱신 조건을 PCWrite | mem_branch_taken으로 처리한다.

```
347 // ID/EX 추가됨!
348 if (mem_flush_all | IDEX_bubble) begin
349     // bubble : 모든 제어신호 0
350     idex_RegWrite <= 0;
351     idex_MemtoReg <= 0;
352     idex_SavePC <= 0;
353     idex_MemRead <= 0;
354     idex_MemWrite <= 0;
355     idex_Branch <= 0;
356     idex_RegDst <= 0;
357     idex_ALUOp <= 0;
358     idex_ALUSrc <= 0;
359     idex_pc_plus4 <= 0;
```

- HAZARD.v

HAZARD 모듈은 데이터 해저드 검출, 스톱, 플러시를 담당한다.

데이터 해저드 검출 로직은 다음과 같다. ID/EX 또는 EX/MEM 단계에 있는 명령어가 IF/ID 단계 명령어의 소스 레지스터(rs 또는 rt)와 동일한 레지스터에 쓰는 경우를 해저드로 판단한다.

MEM/WB → ID 구간의 해저드는 RF 내부 포워딩으로 자동 해결되므로 별도로 검출하지 않는다.

data_hazard = 1일 때 스톱 동작은 다음과 같다: PCWrite = 0으로 PC를 고정하고, IFIDWrite = 0으로 IF/ID 레지스터를 유지하며, IDEX_bubble = 1로 ID/EX에 NOP을 주입한다. 이를 통해 IF/ID 단계의 명령어가 해저드가 해소될 때까지 재 디코딩된다.

플러시 로직: IFID_flush = branch_taken | jump_taken으로, branch 또는 jump가 taken될 때 IF/ID 레지스터를 NOP으로 초기화한다.

```
48
49 // ID/EX 와 충돌? (직전 명령)
50 wire idex_use_rs = (idex_wr_addr == ifid_rs);
51 wire idex_use_rt = (idex_wr_addr == ifid_rt);
52 wire idex_hazard = idex_RegWrite && (idex_wr_addr != 0)
53 | | | | && (idex_use_rs || idex_use_rt);
54
55 // EX/MEM 와 충돌? (두 cycle 전 명령)
56 wire exmem_use_rs = (exmem_wr_addr == ifid_rs);
57 wire exmem_use_rt = (exmem_wr_addr == ifid_rt);
58 wire exmem_hazard = exmem_RegWrite && (exmem_wr_addr != 0)
59 | | | | && (exmem_use_rs || exmem_use_rt);
60
61 wire data_hazard = idex_hazard | exmem_hazard;
62
```